

Operations Runbook

Local setup, one-job execution, provider controls and incident recovery.
Commands grounded in the current repository.

Version 1.0 · 5 October 2026

Source snapshot: f77645a

Contents

1. Operating scope
2. Start the local environment
3. Register and configure a worker
4. Submit and execute one job
5. Provider controls and cancellation
6. Inspect controller state and PostgreSQL
7. Incident playbook
8. Validation commands and safe test setup
9. Shutdown, backup and restart
10. Current deployment boundary and references

1. Operating scope

This runbook covers the current Linux local lab: a Python FastAPI controller, PostgreSQL in Podman/Docker Compose, and explicitly invoked workers. It is designed for the existing Chromebook/Crostini and rootless Podman workflow, while preserving the repository's Docker option.

Run commands from the MeshCompute repository root. Examples use `podman-compose`; if your working setup uses the Compose plugin, use `COMPOSE='podman compose'`. Docker users can use `COMPOSE='docker compose'`, with the Docker-specific engine and image targets. Do not mix engines accidentally: their image stores can differ.

The commands below were checked against source, not executed against your lab during this documentation task. They assume the pinned revision or a compatible later revision. Review changes before applying them to a newer checkout.

2. Start the local environment

First-time Python setup

Requires Linux, Python 3.12+, an installed Compose implementation, and a usable local container engine.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
# Create local controller configuration only if it does not exist.
test -f .env || cp .env.example .env
```

`.env` configures the controller; workers use process environment variables and do not load that file. `MESHCOMPUTE_DATABASE_URL` can override the default database connection.

Podman readiness

```
systemctl --user enable --now podman.socket
systemctl --user status podman.socket --no-pager
make check-engines
```

Socket enablement is a one-time host setup action; it is unnecessary if already configured. The default worker socket is `$XDG_RUNTIME_DIR/podman/podman.sock`, falling back to `/run/user/<uid>/podman/podman.sock`. `make check-engines` diagnoses API usability; a successful probe alone does not prove workload cgroup restrictions can be enforced.

Start and verify

In Terminal A, with the virtual environment active:

```
make dev-up COMPOSE=podman-compose
```

This starts PostgreSQL, waits for it, applies Alembic migrations, and runs the controller in the foreground at `127.0.0.1:8000`. Ctrl-C stops the controller; the database remains running.

In Terminal B:

```
source .venv/bin/activate
curl --fail http://127.0.0.1:8000/health
make dev-status
make dev-state
alembic current
alembic check
```

Expected: health returns `{"status": "ok"}`, controller/database checks succeed, and schema matches the reviewed migration head 0006. `/health` only checks database connectivity; it can pass while migrations are wrong. Use `http://127.0.0.1:8000/docs` for API inspection.

Build example images explicitly

```
make examples-check
make examples-build-podman
```

The examples cover success, failure, sleep, CPU burn, memory hold, Monte Carlo, and a rejected-volume fixture used by execution checks. Test suites do not silently install engines or build images. For Docker use `make examples-build-docker`.

3. Register and configure a worker

Each registration creates a new worker identity. Names are not unique identities. Capture credentials in memory and do not enable shell tracing (`set -x`).

```
export MESHCOMPUTE_WORKER_CONTROLLER_URL=http://127.0.0.1:8000
registration=$(curl --fail --silent --show-error \
  "$MESHCOMPUTE_WORKER_CONTROLLER_URL/v1/workers/register" \
  -H 'Content-Type: application/json' \
  -d '{"name": "local-provider", "agent_version": "0.1.0"}')
export MESHCOMPUTE_WORKER_ID=$(printf '%s' "$registration" | \
  python -c 'import json,sys; print(json.load(sys.stdin)["id"])')
export MESHCOMPUTE_WORKER_TOKEN=$(printf '%s' "$registration" | \
  python -c 'import json,sys; print(json.load(sys.stdin)["token"])')
unset registration
export MESHCOMPUTE_WORKER_CONTAINER_ENGINE=podman
export MESHCOMPUTE_WORKER_CPU_LIMIT=1
export MESHCOMPUTE_WORKER_MEMORY_LIMIT_MB=256
mesh-worker status
```

The token is returned only on registration; the controller stores its hash. The production worker does not persist it. Losing the token currently requires registering a new identity. Before abandoning an old identity, inspect its reservations and labeled containers; a new identity does not automatically clean up the old one's workloads.

Configuration suffix	Default or use
CONTROLLER_URL	<code>http://127.0.0.1:8000</code>
ID, TOKEN	Required UUID and token for agent operations; local controls need the ID but no token
CONTAINER_ENGINE	<code>auto</code> , <code>podman</code> , or <code>docker</code> ; <code>auto</code> prefers usable Podman
CPU_LIMIT, MEMORY_LIMIT_MB	Initial contribution; defaults are zero; memory is MiB
STATE_DIR	<code>\$XDG_STATE_HOME/meshcompute</code> or <code>~/.local/state/meshcompute</code>
PODMAN_SOCKET, DOCKER_SOCKET	Local engine socket overrides; Docker defaults to <code>/var/run/docker.sock</code>

All suffixes use the prefix `MESHCOMPUTE_WORKER_`. Stored provider settings take precedence over initial environment contribution values after first use. To change existing settings, use `mesh-worker resources`, not just a new export. Limits above physical capacity are capped when advertised.

4. Submit and execute one job

Use Terminal B, retaining the worker environment from registration. The example image must exist in the selected engine's store.

```
curl --fail --silent --show-error \
  http://127.0.0.1:8000/v1/jobs \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "architecture-smoke",
    "runtime": "container",
    "image": "localhost/meshcompute-success:dev",
    "command": ["python", "/app/main.py"],
    "resources": {"cpu": 0.5, "memory_mb": 128},
    "timeout_seconds": 30,
    "max_attempts": 2
  }'
mesh-worker work-once
```

Keep the returned job UUID and the attempt UUID printed by the worker. `work-once` performs startup reconciliation, starts its own heartbeats, claims once, renews its lease during preparation/execution, cleans up, reports, and exits. A separate heartbeat process for that identity is unnecessary.

The scheduler chooses the oldest compatible fitting queued job, so this invocation may execute an earlier job if the queue was already populated. Inspect `make dev-state` first if you need an isolated smoke check. If you choose a different image, use its correct executable path rather than assuming `/app/main.py`.

```
make dev-state
job_id=REPLACE_WITH_RETURNED_JOB_UUID
attempt_id=REPLACE_WITH_PRINTED_ATTEMPT_UUID
curl --fail "http://127.0.0.1:8000/v1/jobs/$job_id"
curl --fail "http://127.0.0.1:8000/v1/attempts/$attempt_id"
```

Success means job and attempt are `SUCCEEDED`, exit code is 0, output is inspectable, and the controller allocation is released. A workload exit alone is not sufficient: confirm the controller recorded its result.

Command	What it actually does
<code>mesh-worker</code>	Heartbeats and reconciliation; no automatic claims or execution
<code>mesh-worker claim</code>	Diagnostic reservation only; starts no container and does not renew
<code>mesh-worker work-once</code>	At most one real execution; exits after result or error
<code>mesh-worker reconcile</code>	One reconciliation pass; reports whether active work still blocks claims

A diagnostic claim normally expires after 30 seconds and becomes `LOST`; it consumes an attempt and may requeue the job. Avoid using it as a harmless queue-inspection command.

5. Provider controls and cancellation

Use the same worker ID, local user, engine configuration and state directory as the agent. Local controls work without the worker bearer token.

```
mesh-worker status
mesh-worker pause
mesh-worker drain
mesh-worker resources --cpu 1 --memory 256
mesh-worker stop-all
```

These are independent actions, not a sequence to run blindly. Choose the action you need.

Action	Expected effect
pause	Stop new local claims; existing execution continues
drain	Stop new local claims and let active work finish; participation remains draining until changed
resources	Persist new contribution and advertise it; requires both CPU and integer MiB memory
stop-all	Pause, reclaim live work and explicitly worker-labeled leftovers; wait for local cleanup confirmation
resume	Permit new work if runtime health and capacity allow

Lowering CPU/memory does not currently terminate or resize an active workload. Use `stop-all` when you need immediate reclamation. It is scoped to that worker identity, not every logical identity on the host. If it fails, cleanup is unconfirmed; inspect the engine before resuming.

For requester cancellation of a queued job:

```
curl --fail -X POST \
  "http://127.0.0.1:8000/v1/jobs/$job_id/cancel"
```

This returns 409 once the job is assigned/running. There is no current local `cancel <attempt-id>` command. Provider preemption can requeue a job; requester cancellation ends requester interest, so the two operations are not interchangeable.

6. Inspect controller state and PostgreSQL

Prefer the API and make `dev-state` for routine diagnosis. SQL is useful when comparing persistent state, allocations and leases. In the default Podman Compose lab:

```
podman-compose exec postgres psql -U meshcompute -d meshcompute
```

Useful psql inspection:

```
\dt
\d jobs
\d job_attempts
\d worker_allocations
BEGIN READ ONLY;
SELECT id, name, state, last_heartbeat FROM workers;
SELECT id, name, state, max_attempts FROM jobs;
SELECT id, job_id, worker_id, attempt_number, state,
       lease_expires_at, failure_reason
FROM job_attempts ORDER BY created_at DESC LIMIT 20;
SELECT worker_id, SUM(cpu_reserved) AS cpu_reserved,
       SUM(memory_reserved_mb) AS memory_reserved_mb
FROM worker_allocations GROUP BY worker_id;
ROLLBACK;
\q
```

SQL `workers.state` is the last reported state. The API derives effective health from heartbeat age, so a database row can say `HEALTHY` while the API correctly says `OFFLINE`. Do not select or share token hashes unnecessarily, and do not repair reservations by manually deleting allocation rows: that can disagree with real execution.

Optional desktop database clients can connect to 127.0.0.1:5432, database/user/password meshcompute in the unchanged local Compose configuration. These are development-only defaults.

For engine inspection:

```
podman ps -a --filter label=io.meshcompute.attempt
podman ps -a \
  --filter "label=io.meshcompute.worker=$MESHCOMPUTE_WORKER_ID"
```

Read output tails from the attempt API. Engine logging is deliberately disabled for workloads, so podman logs is not the authoritative MeshCompute result store. Tails are limited to 64 KiB per stream and can be truncated.

7. Incident playbook

A job remains queued or work-once reports no work

Inspect `make dev-state`, `mesh-worker status`, and `/v1/workers`. Confirm fresh effective HEALTHY state, nonzero stored contribution, runtime availability, enough CPU/memory after reservations, and no local pause/drain. REGISTERING means no successful heartbeat. Staleness becomes degraded at 15 seconds and offline after 30.

Run `make check-engines` for runtime problems and `mesh-worker reconcile` for unresolved prior work. A failed startup comparison prevents heartbeats from starting; a successful comparison with existing work can allow heartbeats but still block claims. Do not repeatedly issue diagnostic claims to fix a queue.

Claim response was lost or invalid

The assignment may have committed. Inspect `jobs/attempts/allocations` and reconcile before another claim. Do not blindly retry the HTTP claim or start a guessed assignment. An abandoned valid lease can expire and be recovered normally.

Container execution fails

Inspect the attempt's `failure_reason`, engine and output tails. `IMAGE_PULL_FAILED` suggests an unavailable image or registry path; verify the image exists in the chosen engine. `SECURITY_POLICY_UNSUPPORTED` means policy verification rejected the runtime/image/resources; do not weaken restrictions to force success. Image-declared volumes are rejected. CPU requests below the executor's supported quota minimum (0.01 CPU) or unsupported precision can also be rejected after scheduling.

`NONZERO_EXIT` is a workload error; `TIMEOUT` exceeded the execution deadline. Image preparation has a separate bounded pull deadline. Ordinary failure and timeout do not automatically retry, even if `max_attempts` is larger than one.

Worker crashes or loses controller connectivity

Keep PostgreSQL and the controller available if possible. Lease renewal loss makes a surviving agent stop local execution; a hard kill may leave an orphan container. After lease expiry, controller recovery marks the attempt lost, releases its reservation and requeues while attempts remain. Recovery runs about every five seconds, but database outages/locks and backlog can delay it.

Restart using the same worker identity/configuration, then reconcile. Valid surviving work is not adopted; it blocks claims until resolved. If the provider needs resources immediately, `mesh-worker stop-all` works locally without waiting for the controller. Verify both local container removal and eventual controller release.

Missing workload or leftover container

Run `mesh-worker reconcile`. A successfully scanned missing running workload can become `LOST / WORKLOAD_MISSING`. Unreachable engines are not evidence of absence. Reconciliation preserves unrelated containers and persistent volumes, and leaves unattributable legacy containers for manual inspection. A container name by itself is not sufficient ownership evidence.

Result reporting or cleanup is unresolved

Do not assume capacity was released or launch the same attempt again. Inspect the attempt API, allocation ledger and labeled engine objects. Cleanup failures can leave a workload present; a reporting failure can leave a committed terminal result or an active lease. Once ownership and runtime connectivity are restored, reconcile and let normal result/recovery paths update state. Conflicting terminal reports are rejected rather than overwriting history.

Authentication or protocol rejection

Check controller URL, worker UUID and token pairing. 401 indicates invalid worker credentials; 403 can indicate another worker's attempt; 409 indicates state/lease conflict. Permanent authentication/protocol errors stop the agent rather than retry forever. Never paste tokens into shared logs. If credentials are lost, inspect and retire the old identity's work before registering a replacement.

Controller or database unavailable

Check the foreground controller terminal and database status with `podman-compose ps` and `podman-compose logs --tail=100 postgres`. Restore database availability and required schema, then restart the controller. Persisted leases survive restart and recovery resumes. Existing workers may already have stopped due to lease loss, so expect lost attempts and possible retries; do not infer successful execution from old `RUNNING` rows.

8. Validation commands and safe test setup

Target	What it verifies	Setup and effects
<code>make test-local-runtime</code>	Engine, example runs and cgroup restrictions	Podman and prebuilt images; no scheduler proof
<code>make test-local-assignment</code>	Real-worker advertisement and controller assignment	Controller plus separate <code>make dev-real-worker</code> ; assignment is not execution
<code>make test-local</code>	Runtime then assignment checks	Aggregate of the preceding two only
<code>make test-execution</code>	Real controller-to-worker execution, results and failure paths	Normal controller running; no separate heartbeat agent needed
<code>make test-recovery</code>	Lease renewal/expiry, restart, stale results and retries	Owns temporary controller; stop normal controller/agents first
<code>make test-provider</code>	Local controls, offline reclaim and preemption	Owns temporary controller; needs idle migrated database
<code>make test-reconciliation</code>	Orphan/missing-work reconciliation and ownership safety	Owns temporary controller; needs idle migrated database
<code>make test-scheduler</code>	Scheduling/accounting/concurrency scenarios	Explicit destructive local reset confirmation; disposable lab only

`make test-local` is not the full suite. For the recovery/provider/reconciliation group, stop the normal controller and workers, leave PostgreSQL running, apply migrations, build required images, and inspect state before running one suite at a time:

```
make dev-db-up COMPOSE=podman-compose
make dev-migrate
make dev-state
make test-recovery
make test-provider
make test-reconciliation
make dev-state
```

These suites refuse active queued/running work or unsafe setup and retain result history. Resolve the active work rather than bypassing guards. Scheduler reset tests are separate because they erase local fixture state. Lab helpers use protected credential files and a separate provider-state directory; manual production commands use environment credentials. Do not confuse those identities or directories.

This documentation task did not rerun these tests. Engine support in source and a historical successful lab run are distinct from validation on a newly configured machine.

9. Shutdown, backup and restart

For graceful shutdown, drain each executing worker and let its one-shot job finish. If immediate local reclamation is required, stop-all each identity and confirm cleanup. Stop heartbeat processes, stop the foreground controller, then stop the Compose stack:

```
make dev-down COMPOSE=podman-compose
unset MESHCOMPUTE_WORKER_TOKEN
```

Compose down preserves the named database volume. `down -v`, `make dev-reset`, and migration downgrades can destroy state; they are not routine shutdown or incident-repair actions. A stopped database is not a backup.

Before intentional destructive lab work, an operator can take a logical backup while PostgreSQL is available:

```
mkdir -p backups
chmod 700 backups
umask 077
backup_file="backups/meshcompute-$(date +%Y%m%d-%H%M%S).dump"
podman-compose exec -T postgres \
    pg_dump -U meshcompute -d meshcompute -Fc > "$backup_file"
test -s "$backup_file"
```

This is an operator procedure, not a repository-managed backup service. Check the dump command's exit status; nonempty output alone is insufficient verification. Protect the dump: it contains workload history, outputs and worker token hashes. Restore-test into a separate disposable database before relying on it. Restoring an old snapshot into an active controller can replay stale assignments, so recovery requires workers to be stopped, schema/version compatibility checked, and reconciliation before claims resume.

Restart with `make dev-up`, confirm database/schema health, restore the intended worker environment, inspect controller state and reconcile before running new work. Preserve the existing database volume and provider settings.

10. Current deployment boundary and references

This revision is suited to a trusted local lab. Requester submission, inspection, cancellation and registration lack account-level authorization. There is no production service installer, remote provider dashboard, Prometheus endpoint, managed artifact storage, or validated cloud deployment configuration in the reviewed tree.

Before a cloud pilot, decide how to restrict controller access and authenticate requesters, terminate TLS, protect PostgreSQL, store/rotate credentials, back up and restore state, supervise services, collect logs/metrics, and manage image/disk growth. This is proposed operational work, not existing functionality. Keep the worker's outbound-only connection model and local resource authority intact.

Source baseline: [MeshCompute at f77645a](#). Command definitions: `Makefile` and `worker/main.py`. Detailed lab instructions: `local_test/README.md` and `docs/development.md`. Runtime/configuration: `worker/config.py`, `worker/agent/`, `worker/preemption/control.py`, `worker/executors/`. Controller behavior: `controller/api/` and `controller/services/`.

Maintain this runbook alongside changes to commands, credentials, configuration, migrations, recovery behavior and test prerequisites. Consult the Architecture Guide for state and ownership invariants before changing operational procedures.